

# Fundamentals of Web Applications

## Backend Development: Introduction to Express

Lectures 20 and 21 – May 20 and 21, 2026

Michael Soprano — [michael.soprano@uniud.it](mailto:michael.soprano@uniud.it)

University of Udine — Department of Mathematics,  
Computer Science, and Physics (DMIF)

# Outline

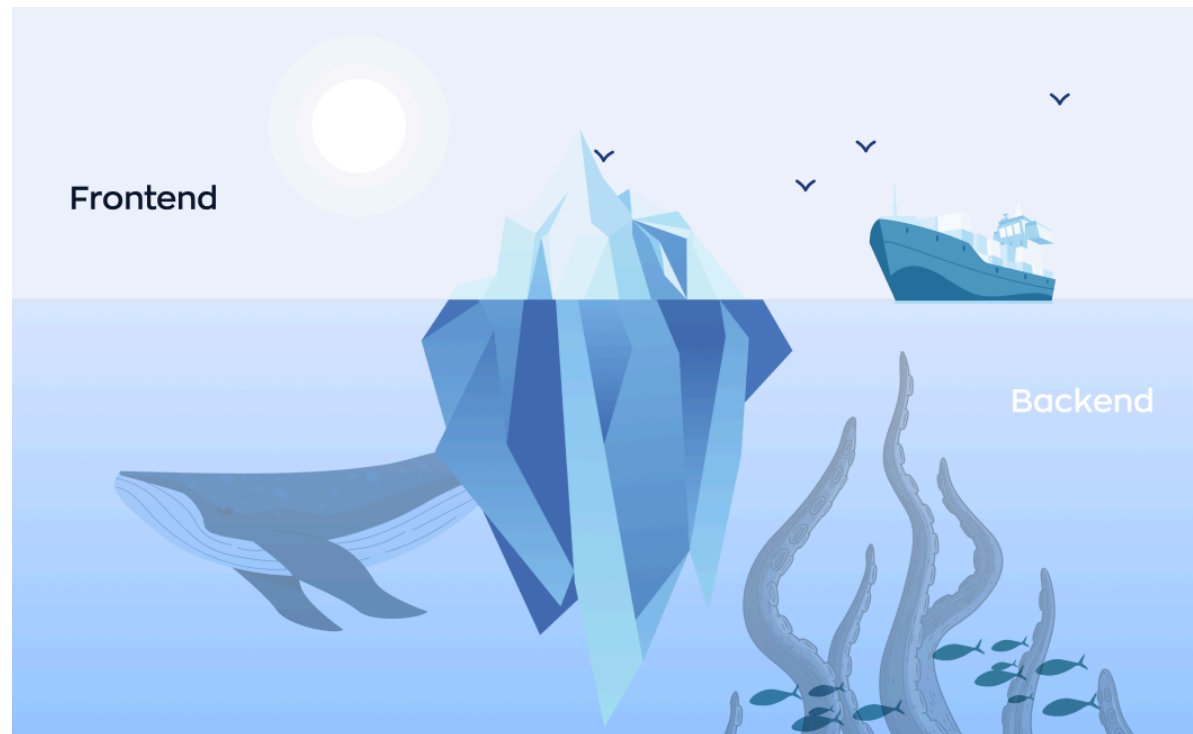
1. Entering the Backend
2. Express as a Server
3. Frontend-to-Backend Requests
4. Routes and Responses
5. JSON API Responses

## Frontend and Backend Roles

- The **frontend** manages the interface, user interaction, and DOM updates
- The **backend** runs on a server and responds to client requests
- The frontend asks for **data** or **actions** when the interface needs them
- The backend applies **application logic** and sends back a response

# Client and Server Flow

- The browser requests an initial **hypermedia document**
- The document loads **CSS** and **JavaScript** as usual
- Client-side JavaScript can send additional **HTTP requests**
- The backend returns content that the frontend can **render** or **process**



# JavaScript Beyond the Browser

- Browser JavaScript works with the **current document**
- It can select elements, handle events, and update the **rendered interface**
- Backend JavaScript runs outside the browser through **Node.js**
- The same language is used in a different **execution environment**

# Different Runtime Capabilities

- The browser provides APIs for the **DOM**, events, forms, and rendering
- Node.js provides APIs for **files**, **processes**, modules, and network servers
- Backend code does not have access to `document` or `window`
- Code must be written for the environment where it **runs**

# Requests Instead of Events

- Frontend code often reacts to **user actions**
- Backend code reacts to **client requests**
- A backend route can compute a response when an **HTTP request** arrives
- The response may contain **text**, **HTML**, **JSON**, files, or status information

# Full-Stack Communication

- A **full-stack application** combines browser code and server-side code
- The frontend manages the **interface** and sends requests when it needs data or actions
- The backend receives those requests, applies **application logic**, and sends responses
- The two sides remain separate, but communicate through **HTTP**

# Toward a Backend Server

- A backend needs a program that can stay **running**
- The program must listen for incoming **HTTP requests**
- It must decide which response to send for each requested **path**
- Express provides a compact way to define this **server-side behavior**

## Runtime Environment: Node.js

- **Node.js** is an environment for executing JavaScript outside the browser
- It can run JavaScript files directly from the **terminal**
- It provides server-side capabilities such as **files** and **network access**
- Express applications run on top of **Node.js**



# Installing Node.js

- Node.js must be installed before running **Node.js programs**
- The `node` command runs JavaScript files from the terminal
- The `npm` command installs project **dependencies**
- Installing Node.js usually also installs **npm**
- Use the official installer or a package manager for the **operating system**

```
node -v  
npm -v
```

# Running JavaScript with Node.js

- Browser JavaScript runs when the browser loads a **hypermedia document**
- Node.js runs JavaScript files as independent **programs**
- A backend project usually starts from a file such as `server.js`
- The command `node server.js` starts the **server program**

```
node server.js
```

# Project Dependencies: `npm`

- Backend projects often use external **packages**
- Start by creating and entering the **project folder**
- `npm init -y` creates the initial **package.json** file
- `npm install express` installs Express and records it as a **dependency**
- npm stores installed packages in `node_modules/` and exact dependency versions in `package-lock.json`

```
mkdir AA_static-server
cd AA_static-server

npm init -y
npm install express
```

# Project Metadata: `package.json`

- `package.json` describes the **Node.js project**
- It stores the project name, version, scripts, and dependencies
- The `scripts` section defines commands that can be run with **npm**
- The `dependencies` section lists packages required by the **application**

```
{
  "name": "aa_static-server",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

# Server Frameworks

- A **server framework** provides a structure for building backend applications
- It helps connect incoming **requests** to the code that should handle them
- Developers focus on the **application-specific logic** for each route
- The framework manages many repetitive details of **HTTP communication**

# Server Framework: Express

- **Express** is a server framework for Node.js
- It helps define how a server responds to **HTTP requests**
- An Express application can serve files and compute **dynamic responses**
- The server stays active and waits for **clients** to connect

express

# Express Application

- An Express server starts from an **application object**
- The application object stores the **server configuration**
- Routes and static file rules are registered on this object
- Starting the application makes it listen for **incoming requests**

```
const express = require("express")  
  
const app = express()
```

# Listening for Requests

- A server must listen on a **port**
- The port identifies where the application accepts **connections**
- During development, a common choice is `3000`
- `localhost` refers to the **current machine**

```
const port = 3000

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

# First Express Server

- The application imports **Express**
- It creates an **app** object
- It starts listening on a local **port**
- The server is running, but no custom routes have been defined yet

```
const express = require("express")

const app = express()
const port = 3000

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

# Running the Server

- Open a terminal in the chosen **folder**
- Start the server with **Node.js**
- The terminal keeps showing the running **process**
- Stop the server with `Ctrl + C`

```
node server.js
```

```
Server running at http://localhost:3000
```

# Multiple Terminals

- The server process keeps the current **terminal busy**
- In editors such as VS Code, more than one terminal can be opened
- One terminal can keep the **server running**
- Another terminal can be used for commands while the server remains **active**

```
Terminal 1  
node server.js
```

```
Terminal 2  
ls  
npm install
```

## Static File Serving: `express.static()`

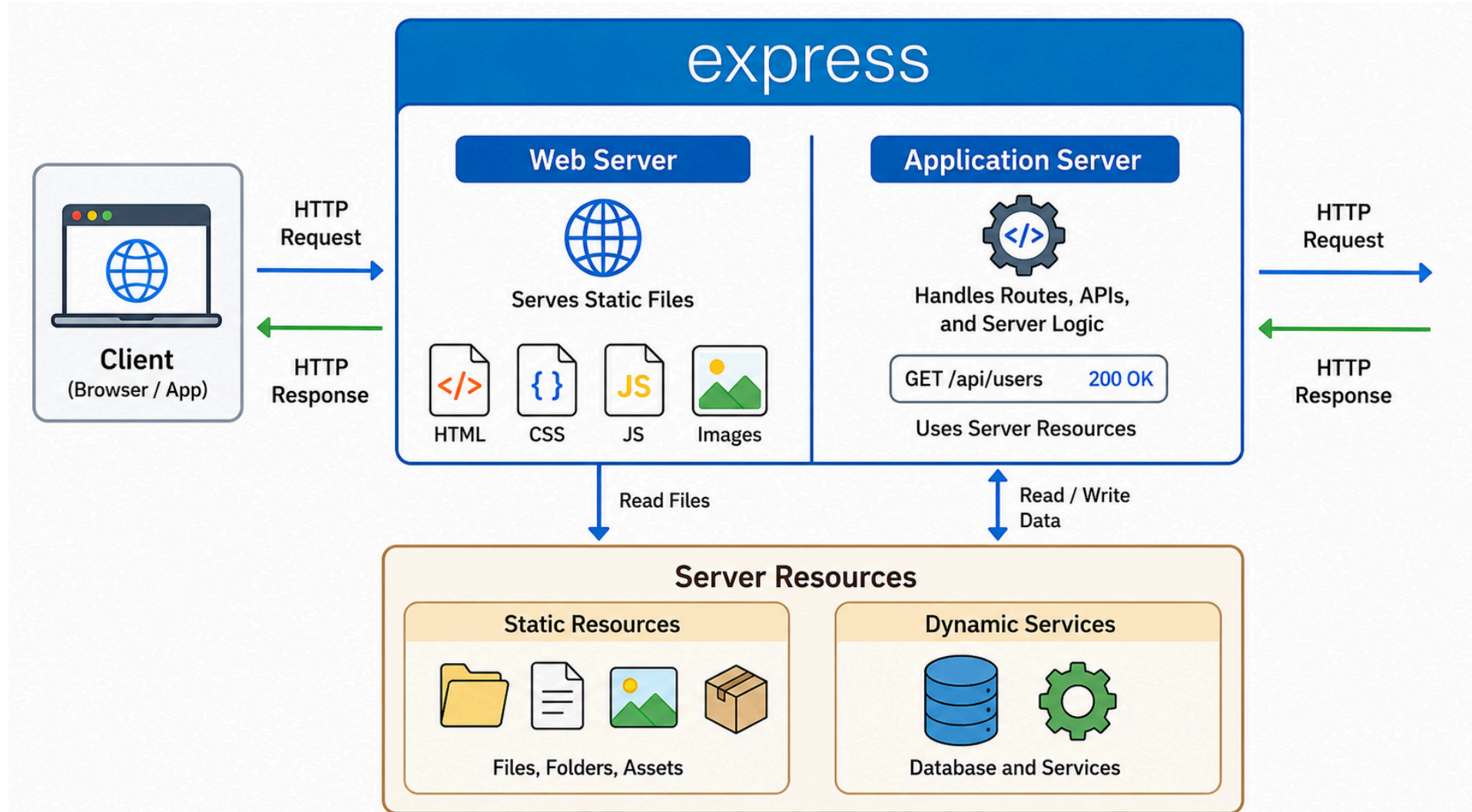
- A backend can serve **frontend files** as static assets
- Static assets are existing files returned **as they are**
- `express.static()` maps a project folder to public **HTTP paths**
- Files inside `public/` can be requested directly by the **browser**

```
app.use(express.static("public"))
```

# Static Files and Dynamic Routes

- Express can deliver **static frontend files** from the `public` folder
- The same application can also define **dynamic routes**
- Static file serving sends files without route-specific **server-side logic**
- Route handling runs server-side code and computes **responses**

# Express Server Roles



Express can serve static files as a web server and handle dynamic routes as an application server

## Example: Source Code

- The files can be found in the `Samples/Backend/` folder
- Sample **resource**: `AA_static-server/`
- The server exposes a **public** folder with frontend assets
- The browser receives `index.html`, CSS, and JavaScript from **Express**

# Example: Project Layout

- The server code stays in `server.js`
- The frontend files stay inside the **public** folder
- `package.json` records the dependency on **Express** and the start command
- `package-lock.json` records the exact installed **dependency versions**
- `node_modules/` is generated by npm and contains the installed **packages**

```
AA_static-server/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── server.js  
├── package.json  
└── package-lock.json
```

# Example: Running the Sample

- Move into the example **folder**
- `npm install` downloads the dependencies listed in `package.json`
- `npm start` runs the server through the `start` script
- The `console.log()` message appears in the **terminal** running the server
- Then open `http://localhost:3000/` in the browser

```
cd AA_static-server
```

```
npm install  
npm start
```

```
Server running at http://localhost:3000
```

# Example: Server Code

- Sample **resource**: `AA_static-server/server.js`
- Express is imported and used to create the **application object**
- The `public` folder is exposed through `express.static()`
- The application listens for requests on port `3000`

```
const express = require("express")

const app = express()
const port = 3000

app.use(express.static("public"))

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

# Example: Static Document

- Sample **resource**: `AA_static-server/public/index.html`
- The document loads its stylesheet and script from the same **server**
- The script is deferred so the document can be **parsed first**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Static Express Server</title>
  <link rel="stylesheet" href="style.css">
  <script src="client.js" defer></script>
</head>
<body>
  <main class="page">
    <h1>Static Express Server</h1>
    <p class="message">This document was served by Express.</p>
  </main>
</body>
</html>
```

# Example: Compact CSS

- Sample **resource**: `AA_static-server/public/style.css`
- The CSS styles the static document served by **Express**
- The ready state is applied after the browser runs **client.js**

```
body {  
  font-family: Arial, Helvetica, sans-serif;  
  line-height: 1.5;  
}  
  
.page {  
  max-width: 32rem;  
  margin: 2rem auto;  
}  
  
.message.is-ready {  
  color: #1f4e79;  
  font-weight: 700;  
}
```

# Example: Client Script

- Sample **resource**: `AA_static-server/public/client.js`
- The script runs in the **browser**
- Express only delivers the file to the **client**
- The DOM update still happens on the **frontend**

```
const messageElement = document.querySelector(".message")  
  
messageElement.textContent = "The frontend JavaScript file was loaded successfully."  
messageElement.classList.add("is-ready")
```

# Example: Static Frontend Flow

- The browser requests `http://localhost:3000/`
- Express looks inside the static **public** folder
- `index.html` is returned as the initial **hypermedia document**
- The browser then requests the linked CSS and JavaScript **files**

```
GET /  
  → public/index.html
```

```
GET /style.css  
  → public/style.css
```

```
GET /client.js  
  → public/client.js
```

## Example: Result

- Express serves the **static frontend files**
- The browser renders the HTML and applies the CSS
- The deferred script runs in the browser and updates the **DOM**
- No dynamic backend route has been defined yet

Static Express Server

**The frontend JavaScript file was loaded successfully.**

## Browser Requests: `fetch()`

- Browser JavaScript cannot directly call a **backend function**
- The frontend communicates with the backend through **HTTP requests**
- `fetch()` is the standard browser API for sending those requests
- The requested URL can point to a **backend route**

```
fetch("/message")
```

# Request Target and Method

- `fetch("/message")` sends an HTTP **GET request** by default
- The path `/message` identifies the requested **backend path**
- A relative path is resolved from the same **origin** as the document
- Express must define a route that matches the requested **method** and **path**

```
GET /message
```

# Delayed Results

- A request needs time to reach the server and receive a **response**
- `fetch()` does not return the response body directly
- It returns a **Promise** that resolves when the HTTP response arrives
- `.then()` registers code that should run when that result **arrives**

```
fetch("/message")  
  .then(response => response.text())
```

# Fetch Response Flow

- The first handler receives a **Response object**
- The response body must be read with `.text()`, `.json()`, or another body-reading method
- Reading the body is also **asynchronous**
- The next handler receives the extracted **body value**

```
fetch("/message")
  .then(response => response.text())
  .then(text => {
    messageElement.textContent = text
  })
```

# Handler Parameters

- `response` and `text` are **parameters** of handler functions
- Their names are chosen by the developer
- `response` refers to the HTTP **Response object**
- `text` refers to the extracted **body value**

```
fetch("/message")  
  .then(response => response.text())  
  .then(text => {  
    messageElement.textContent = text  
  })
```

# Inline and Named Handlers

- Inline arrow functions keep short flows **compact**
- Named functions make longer flows more **readable**
- Both forms express the same asynchronous **chain**
- Each handler receives the value produced by the **previous step**

```
function readText(response) {  
  return response.text()  
}  
  
function showMessage(text) {  
  messageElement.textContent = text  
}  
  
fetch("/message")  
  .then(readText)  
  .then(showMessage)
```

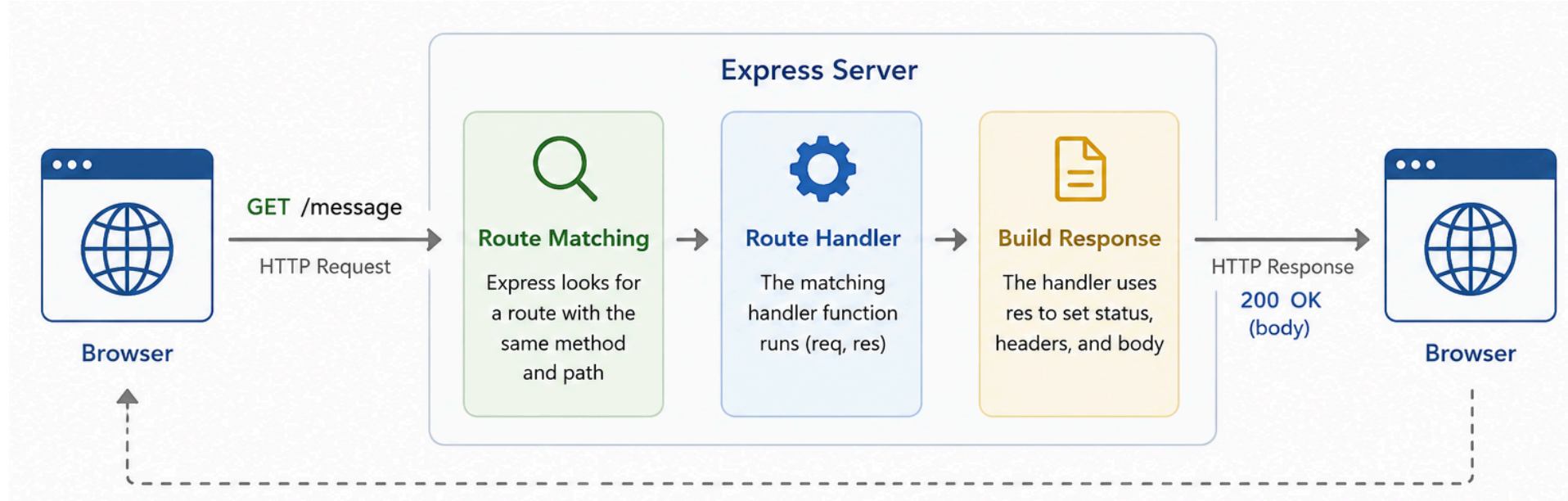
# Handling Request Failures

- A frontend request can fail because of a **network problem**
- Response processing can also fail while reading or using the **body**
- `.catch()` registers code for errors in the asynchronous chain
- Error handling keeps the interface from remaining in a silent **waiting state**

```
fetch("/message")
  .then(response => response.text())
  .then(showMessage)
  .catch(() => {
    messageElement.textContent = "The message could not be loaded."
  })
```

# Routes as Request Handlers

- A **route** connects an incoming request to server-side logic
- Express matches requests by **method** and **path**
- A request sent with `fetch()` must match one of these **routes**
- When a match is found, the route **handler** builds the response



Express matches a request by method and path, then runs the corresponding handler

# Route Method and Path

- `app.get()` registers a route for HTTP **GET** requests
- `"/message"` is the **path** matched by the route
- A request to `GET /message` runs the matching **handler function**
- Other methods or paths require different **routes**

```
app.get("/message", sendMessage)
```

# Handler Function: `req` and `res`

- The handler function receives a **request object** and a **response object**
- `req` describes the request sent by the client
- `res` provides methods for building the response
- The function decides what the server **sends back**

```
function sendMessage(req, res) {  
  res.send("Hello from the Express backend.")  
}
```

## Sending Text: `res.send()`

- `res.send()` sends a response body to the client
- Express sets an appropriate **content type** for simple text
- Sending a response completes the request-response **cycle**
- Each request should receive one clear **response**

```
res.send("Hello from the Express backend.")
```

# HTTP Status Codes

- A **status code** summarizes the outcome of an HTTP response
- `2xx` codes indicate **successful** responses
- `4xx` codes indicate a problem with the **client request**
- `5xx` codes indicate a problem on the **server side**

```
res.status(200).send("Message sent successfully.")  
res.status(404).send("Message not found.")  
res.status(500).send("Internal server error.")
```

## Response Status: `res.status()`

- Express sends `200 OK` by default for successful simple responses
- `res.status()` sets the status code before sending the **body**
- The response is still completed by methods such as `res.send()`
- Explicit status codes make server behavior easier to **inspect**

```
res.status(200).send("Hello from the Express backend.")
```

# Missing Routes and 404

- A request may not match any defined **route**
- In that case, the server cannot return the requested resource
- 404 Not Found tells the client that the target was not found
- Status codes help the frontend interpret what **happened**

```
app.use((req, res) => {  
  res.status(404).send("Resource not found.")  
})
```

# Response Status in the Frontend

- `fetch()` receives an HTTP response even when the status is not **successful**
- The `ok` property helps check whether the response represents a successful **outcome**
- Frontend code can inspect the status before reading the **body**
- Larger API examples will use this pattern more **systematically**

```
function checkStatus(response) {  
  if (!response.ok) {  
    throw new Error("Request failed")  
  }  
  
  return response  
}
```

# Example: Source Code

- Sample **resource**: `AB_fetch-message/`
- `server.js` defines a backend **route**
- `client.js` calls that route with `fetch()`
- The example connects a static frontend to a dynamic **server response**

```
AB_fetch-message/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── server.js  
└── package.json
```

# Example: Server Route

- Sample **resource**: `AB_fetch-message/server.js`
- Static assets are still served from the **public** folder
- `GET /message` returns a response from the **server**
- `res.status(200).send()` sends successful text back to the browser

```
const express = require("express")

const app = express()
const port = 3000

app.use(express.static("public"))

app.get("/message", (req, res) => {
  res.status(200).send("Hello from the Express backend.")
})

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

# Example: Static Document

- Sample **resource**: `AB_fetch-message/public/index.html`
- The page contains a placeholder for the server **message**
- The deferred script will request the message after the document is **parsed**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Fetch Message</title>
  <link rel="stylesheet" href="style.css">
  <script src="client.js" defer></script>
</head>
<body>
  <main class="page">
    <h1>Backend Message</h1>
    <p class="message">Waiting for the backend...</p>
  </main>
</body>
</html>
```

# Example: Compact CSS

- Sample **resource**: [AB\\_fetch-message/public/style.css](https://AB_fetch-message/public/style.css)
- The initial message appears in a **neutral style**
- The ready class marks content received from the **backend**

```
body {  
  font-family: Arial, Helvetica, sans-serif;  
  line-height: 1.5;  
}  
  
.page {  
  max-width: 32rem;  
  margin: 2rem auto;  
}  
  
.message.is-ready {  
  color: #1f4e79;  
  font-weight: 700;  
}
```

# Example: Client Script

- Sample **resource**: `AB_fetch-message/public/client.js`
- `fetch("/message")` calls the backend **route**
- `checkStatus()` verifies whether the HTTP response is **successful**
- `.text()` reads the response body as **plain text**

```
const messageElement = document.querySelector(".message")

function checkStatus(response) {
  if (!response.ok) {
    throw new Error("Request failed")
  }
  return response
}

function showMessage(text) {
  messageElement.textContent = text
  messageElement.classList.add("is-ready")
}

function showError() { messageElement.textContent = "The message could not be loaded." }

fetch("/message").then(checkStatus).then(response => response.text()).then(showMessage).catch(showError)
```

# Example: Request Flow

- The document is loaded from the static **public** folder
- The script sends a second request to the backend **route**
- Express computes a successful response for `/message`
- The browser updates the DOM with the received **text**

```
GET /  
  → public/index.html
```

```
GET /style.css  
  → public/style.css
```

```
GET /client.js  
  → public/client.js
```

```
GET /message  
  → 200 OK  
  → "Hello from the Express backend."
```

## Example: Result

- The frontend was delivered as **static files**
- The message was produced by a backend **route**
- The browser received the response through `fetch()`
- JavaScript updated the DOM without reloading the **document**

Backend Message

**Hello from the Express backend.**

## Structured Data

- Text responses are enough for simple **messages**
- Frontend code often needs several related **values**
- JavaScript already works with arrays and **objects**
- JSON lets the backend send structured data that the browser can **process**

# JSON Responses

- **JSON** is a text format for structured data
- It represents values such as strings, numbers, booleans, arrays, and objects
- It is widely used for frontend-backend **data exchange**
- JavaScript can convert JSON responses into ordinary **objects**

## Sending JSON: `res.json()`

- `res.json()` sends a JavaScript value as a **JSON response**
- Express sets the appropriate **content type**
- Objects and arrays can be sent directly from a route
- The frontend can read the response as structured **data**

```
app.get("/course", (req, res) => {  
  res.status(200).json({  
    title: "Fundamentals of Web Applications",  
    topic: "Introduction to Express",  
    format: "Backend development"  
  })  
})
```

## Reading JSON: `response.json()`

- `response.json()` reads the response body as **JSON**
- The resulting value becomes a JavaScript **object** or **array**
- Frontend code can access properties and render them in the **DOM**
- The JSON parsing step is also **asynchronous**

```
fetch("/course")
  .then(checkStatus)
  .then(response => response.json())
  .then(course => {
    titleElement.textContent = course.title
    topicElement.textContent = course.topic
  })
```

# Example: Source Code

- The files can be found in the `Samples/Backend/` folder
- Sample **resource**: `AC_json-response/`
- `server.js` defines a route that sends **JSON**
- `client.js` reads that response with `response.json()`

```
AC_json-response/  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── server.js  
└── package.json
```

# Example: Server Route

- Sample **resource**: `AC_json-response/server.js`
- `GET /course` returns a successful **JSON response**
- `res.status(200).json()` sends the object to the browser

```
const express = require("express")
const app = express()
const port = 3000

app.use(express.static("public"))

app.get("/course", (req, res) => {
  res.status(200).json({
    title: "Fundamentals of Web Applications",
    topic: "Introduction to Express",
    format: "Backend development"
  })
})

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

# Example: Static Document

- Sample **resource**: [AC\\_json-response/public/index.html](AC_json-response/public/index.html)
- The document contains empty elements for server **data**
- JavaScript fills those elements after the JSON response **arrives**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>JSON Response</title>
  <link rel="stylesheet" href="style.css">
  <script src="client.js" defer></script>
</head>
<body>
  <main class="page">
    <h1 class="course-title">Loading course...</h1>
    <p class="course-topic"></p>
    <p class="course-format"></p>
  </main>
</body>
</html>
```

# Example: Client Script

- Sample **resource**: `AC_json-response/public/client.js`
- `fetch("/course")` requests structured **data**
- `response.json()` parses the JSON response
- The frontend reads object properties and updates the **DOM**

```
const titleElement = document.querySelector(".course-title")
const topicElement = document.querySelector(".course-topic")
const formatElement = document.querySelector(".course-format")

function checkStatus(response) {
  if (!response.ok) { throw new Error("Request failed") }
  return response
}

function showCourse(course) {
  titleElement.textContent = course.title
  topicElement.textContent = `Topic: ${course.topic}`
  formatElement.textContent = `Format: ${course.format}`
}

function showError() { titleElement.textContent = "The course data could not be loaded." }
fetch("/course").then(checkStatus).then(response => response.json()).then(showCourse).catch(showError)
```

# Example: Request Flow

- The browser loads the static **frontend**
- The script sends an additional request to the backend **route**
- Express returns a successful JSON **response**
- The frontend renders selected values in the **document**

```
GET /  
  → public/index.html  
  
GET /style.css  
  → public/style.css  
  
GET /client.js  
  → public/client.js  
  
GET /course  
  → 200 OK  
  → {  
    "title": "Fundamentals of Web Applications",  
    "topic": "Introduction to Express",  
    "format": "Backend development"  
  }
```

# Example: Result

- The backend returns several related **values**
- The frontend receives them as one JavaScript **object**
- Each property can be placed in a different part of the **interface**
- JSON makes the response easier to extend than plain **text**

**Fundamentals of Web Applications**

**Topic:** Introduction to Express

**Format:** Backend development

# Summary

- Express can serve static files and handle dynamic **routes**
- `fetch()` lets frontend JavaScript send HTTP requests to the **backend**
- Routes match requests by HTTP **method** and **path**
- Handlers use `res` to send text, status codes, or **JSON**
- JSON responses let the frontend receive structured **data**
- Multiple routes can expose different pieces of application **data**
- The next step is designing those routes as a coherent **resource-oriented API**

# Bibliography

- Holmes, E., & Harms, D. (2016). *Express in Action*. Manning Publications
  - Chapters 1-3: Introducing Express, Routing, and Responses
- Express.js Official Docs. *Getting Started*
  - <https://expressjs.com/>
- Express.js Official Docs. *Serving static files in Express*
  - <https://expressjs.com/en/starter/static-files.html>
- Express.js Official Docs. *Basic routing*
  - <https://expressjs.com/en/starter/basic-routing.html>

# Bibliography (cont.)

- Express.js Official Docs. *API Reference: Response*
  - <https://expressjs.com/en/api.html#res>
- Node.js Documentation. *Introduction to Node.js*
  - <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- Node.js Downloads
  - <https://nodejs.org/en/download/>
- npm Docs. *About npm*
  - <https://docs.npmjs.com/about-npm>

# Bibliography (cont.)

- MDN Web Docs. *Using the Fetch API*
  - [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)
- MDN Web Docs. *Promise*
  - [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Promise](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise)
- MDN Web Docs. *HTTP response status codes*
  - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>